



7

13

1950 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية



Traverse in Graph (DFS, BFS)

د. مدحت الصّوص

RB Informatics; 31/05/2026

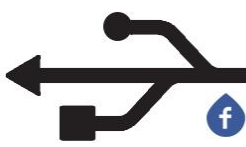
محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

تناولنا في المحاضرة السابقة مفهوم ال Graph و هي مجموعة من العقد و
الوصلات التي تتصل فيما بينها دون أي قيود، و مثلنا ال Graph بطريقتين:
المصفوفة الثنائية و المصفوفة الأحادية والتي كل عنصر فيها يشكّل
LinkedList.

و اليوم سنتحدث عن أهم الخوارزميات التي تُطبّق على ال Graph و هي:
طرق التّجوال

أهم العناوين الواردة في المحاضرة	
رقم الصفحة	عنوان الفقرة
2	طرق التجوال داخل ال Graph



لماذا نتجول داخل الـ Graph؟

لطباعة العقد، أو للبحث عن عقدة معينة.

أثناء التجوال قد لا نمرّ على كافة العقد، يحدث ذلك عندما يوجد لدينا عقد لا تتصل بأيّ وصلة مع عقد أخرى.

عند المرور على جميع العقد في البيان نسمّي العملية (مسح الـ Graph)

ليس من الضروري المرور على جميع العقد في خوارزميات التجوال، كما أنّ العقدة تتمّ زيارتها لمرة واحدة فقط.

عملية المسح بالـ tree كانت أسهل بسبب وجود الجذر الذي نتقل منه نحو الأبناء ثمّ أبناء الأبناء و هكذا...

أمّا في الـ Graph فإنّ عملية المسح أعقد بسبب عدم وجود جذر، و لأنّه يمكنني الوصول لنفس العقدة بأكثر من طريقة (من أكثر من مكان)، لذلك فإنّ طرق التجوال تحاول مسح الـ Graph أو جزء منه، لأنّها تنطلق من عقدة معينة مختارة و نبدأ بالتّنقّل.

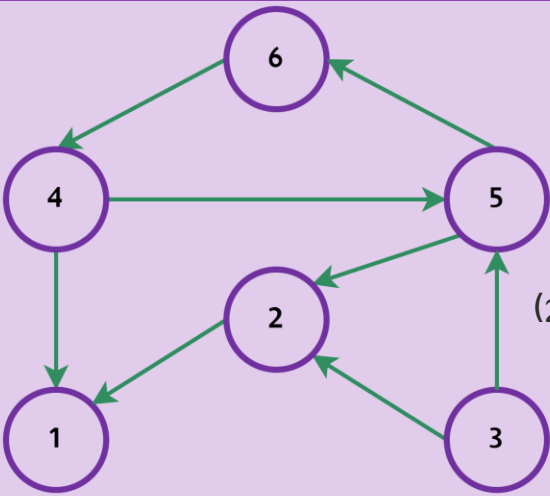
التجوال بطريقة المسح بالعمق (Depth First Search (DFS)

تخيّل نفسك في متاهة وتبحث عن مخرج. الاستراتيجية هنا هي: "امش في أيّ طريق تجده أمامك حتى تصل إلى نهايته (طريق مسدود)، ثم ارجع خطوة إلى الوراء وابحث عن طريق آخر". هذا هو بالضبط "البحث بالعمق".

الخوارزمية تغوص في أعماق المخطط لأسفل نقطة ممكنة قبل أن تبدأ بالرجوع (Backtracking) لاستكشاف الفروع الأخرى.

تعتمد DFS بشكل أساسي على المكدّس (Stack) سواء بشكل صريح، أو بشكل ضمني عبر استدعاء الدوال الذاتي Recursion، المكدّس يتبع مبدأ (LIFO (Last In, First Out.

مثال: ليكن لدينا الـ Graph المجاور:



- لنبدأ الخوارزمية من العقدة (3) أولاً.
- نختار مجاوراً عشوائياً وليكن الـ (2).
- بعدها نختار أيضاً مجاور عشوائي لـ (2) وهو الـ (1).
- بعد زيارة العقدة (1) نلاحظ أنه لا يمكننا انتقال إلى أي عقدة أخرى، أي أننا وصلنا إلى نهاية الطريق، لذلك نترجع إلى العقدة السابقة (2) ونبحث عن مجاور عشوائي آخر لنكمل فيه مسح الـ Graph.
- نجد أنه ليس للعقدة 2 مجاور سوى العقدة التي زرتها وهي الـ (1)، لذلك نترجع خطوة أخرى إلى الوراء لنعود إلى العقدة (3).
- نلاحظ أن (3) تمتلك مجاور لم تتم زيارته وهو العقدة (5).
- ننتقل للعقدة (5)، و لديها مجاوران هما الـ (6) والـ (2)، و لكن الـ (2) قد قمنا بزيارتها مسبقاً؛ لذلك لا نذهب إليها وننتقل إلى (6).
- بعد الـ (6) ننتقل إلى (4) لأنه لا يوجد مجاور آخر لها.
- عند الوصول إلى العقدة (4) نلاحظ أن كل مجاوراتها قد قمنا بزيارتها مسبقاً وهما الـ (1) والـ (5) لذلك نتوقف هنا ونكون قد مسحنا الـ Graph كاملاً.

تذكر:

الخوارزمية السابقة ليس من الضروري أن تمسح جميع عناصر الـ Graph، ففي المثال السابق إذا بدأنا الخوارزمية من العقدة (1) لن نستطيع التحرك من مكاننا لأنه لا يوجد أي مجاور لها نستطيع الانطلاق منه و التّجول على عقد الـ Graph.

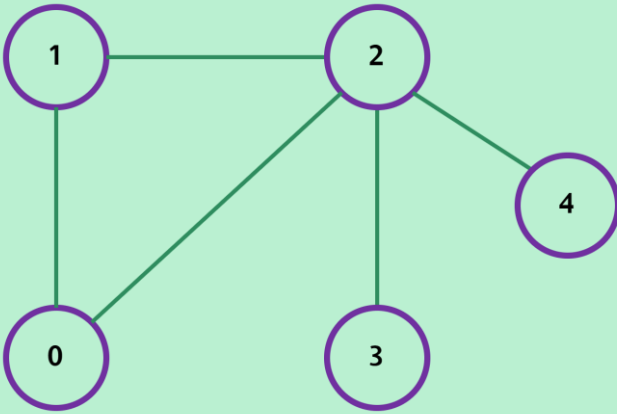
Pseudocode Implementation:

```

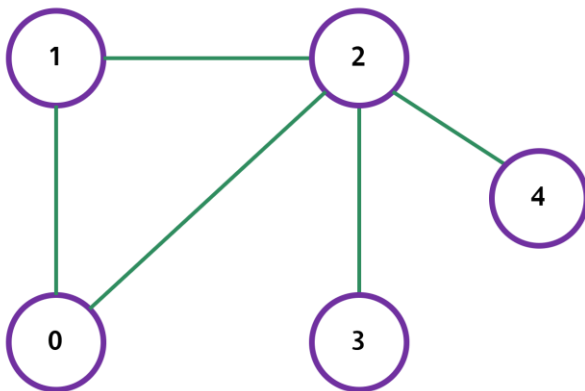
1  boolean visited[n] = false;
2  void DFS(Boolean G[][], int node){
3      visited[node] = true;
4      print (node);
5      for (i=1 → n)
6          if(G [node][i] == true and visited[i] == false)
7              DFS(G, i);
8  }
```

مثال توضيحي:

بالنظر إلى الرسم البياني (Graph) التالي،
أوجد ترتيب المرور عبر العقد باستخدام خوارزمية البحث بالعمق
(DFS Traversal).



قم بإنشاء مصفوفة للعقد المُزارَة [visited] لتتبع العقد التي تمّ استكشافها وتهيئة قيمها ب False (اختصاراً سنكتب F) بالإضافة إلى تجهيز مصفوفة النتيجة [res] ومكدّس الاستدعاءات (Call Stack).

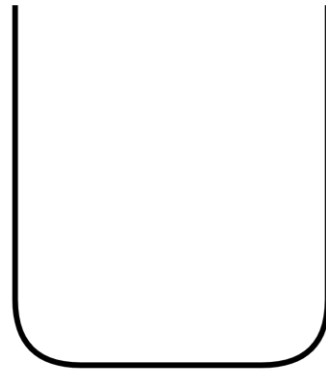


visited[] =

0	1	2	3	4
F	F	F	F	F

res[] =

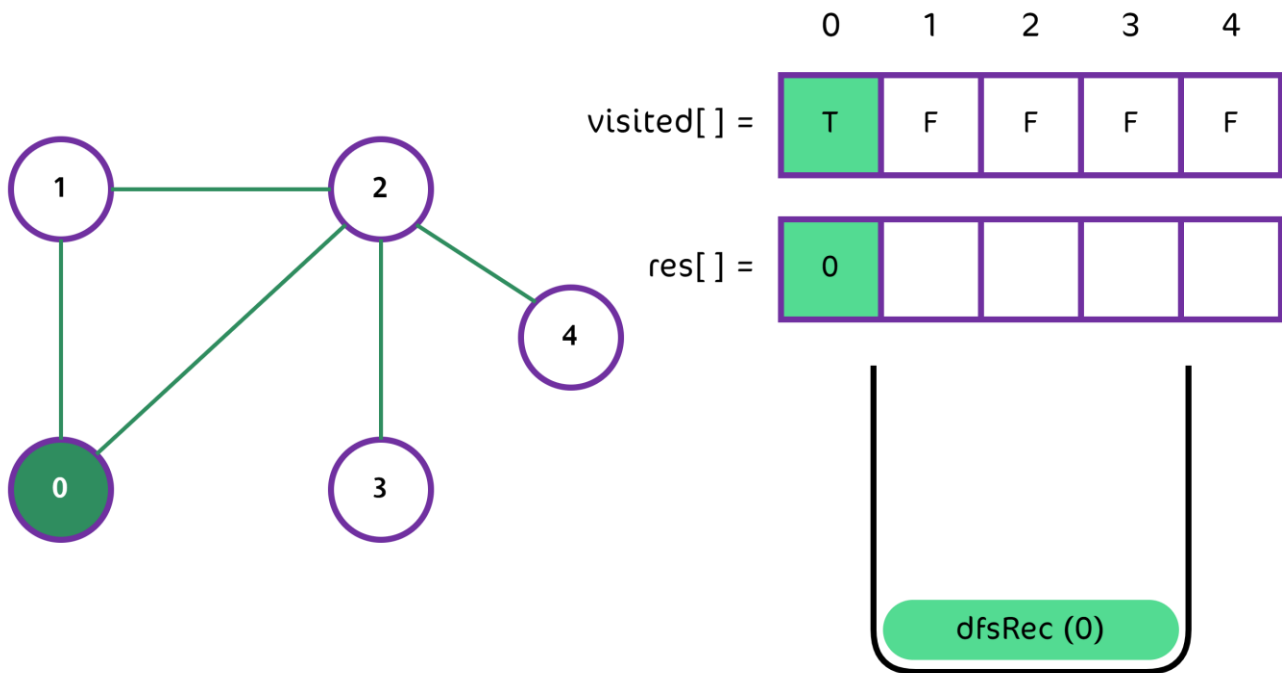
0	1	2	3	4



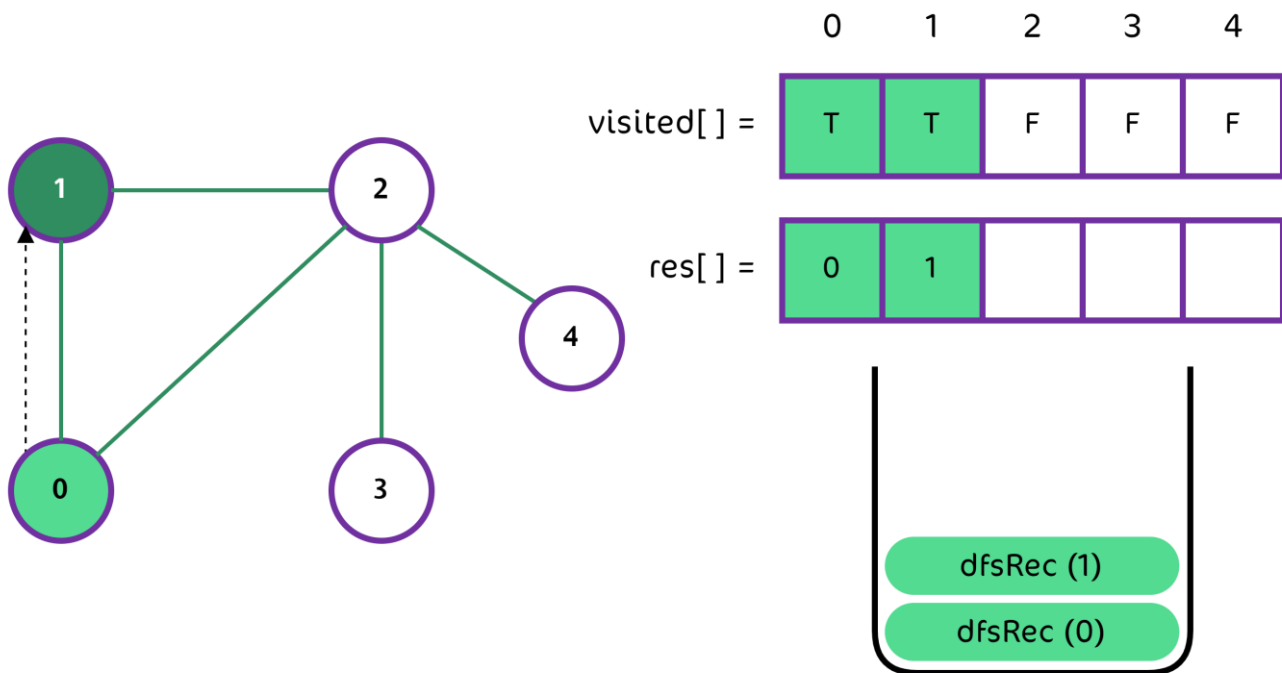
Work silently,...
And let success make the noise



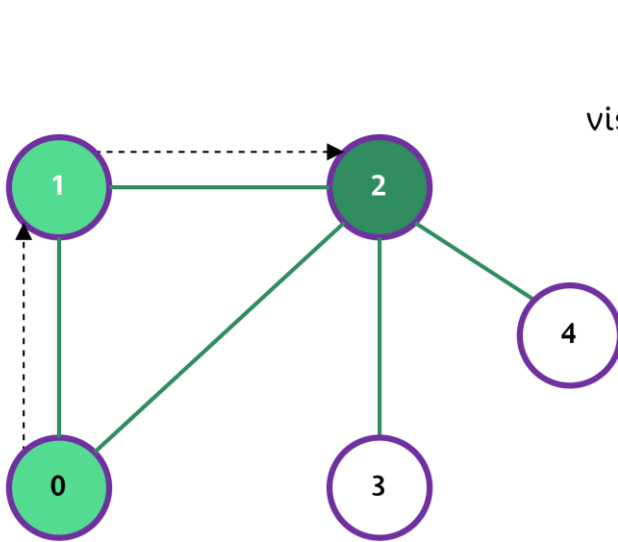
نبدأ من عقدة المصدر (الرأس 0)، ويتم استدعاء الدالة العودية لها $\text{dfsRec}(0)$ ، وتعديل قيمتها في المصفوفة المُرارة إلى True (اختصاراً سنكتب T)، وإضافتها للنتيجة.



ننتقل لزيارة العقدة المجاورة للعقدة 0، وهي العقدة 1. يتم استدعاء الدالة لها $\text{dfsRec}(1)$ ، وتعليمها كعقدة مُزارَة وإضافتها إلى النتيجة.



ننتقل لزيارة العقدة المجاورة للعقدة 1، وهي العقدة 2. يتم استدعاء الدالة لها $\text{dfsRec}(2)$ ، وتعليمها كعقدة مُزارَة وإضافتها إلى النتيجة.

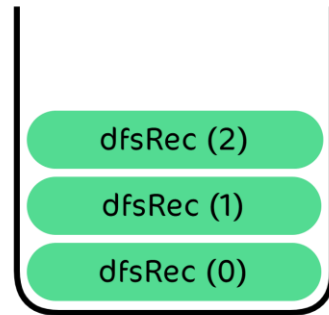


visited[] =

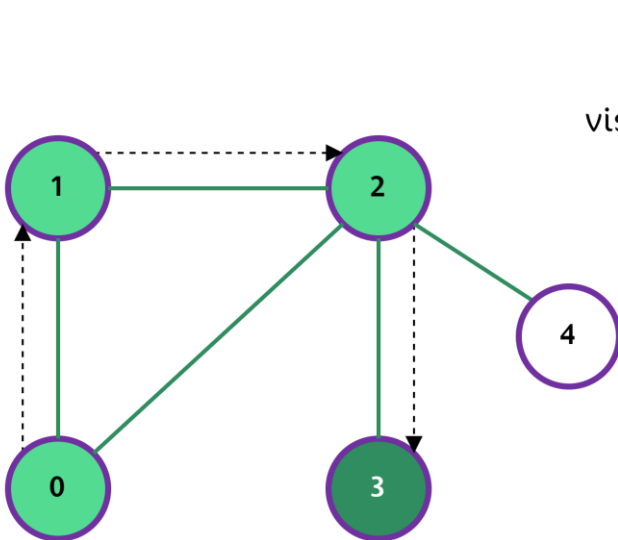
0	1	2	3	4
T	T	T	F	F

res[] =

0	1	2		
---	---	---	--	--



ننتقل لزيارة العقدة المجاورة للعقدة 2، وهي العقدة 3. يتم استدعاء الدالة لها $\text{dfsRec}(3)$ ، وتعليمها كعقدة مُزارَة وإضافتها إلى النتيجة.



visited[] =

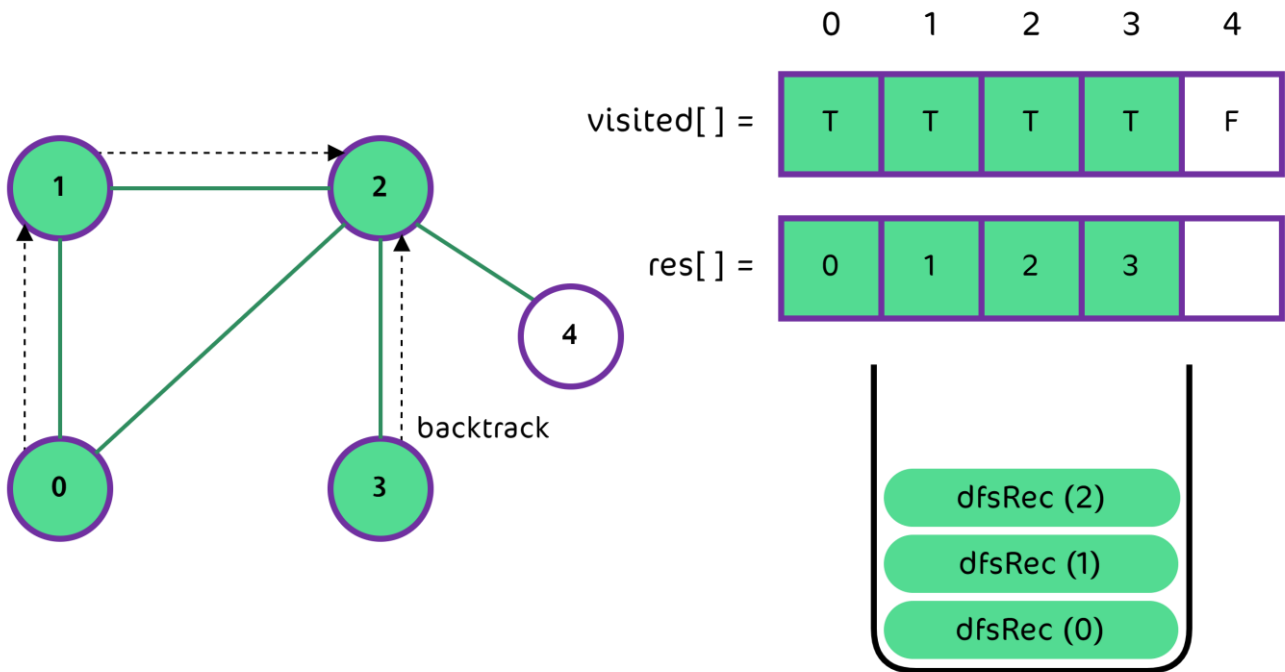
0	1	2	3	4
T	T	T	T	F

res[] =

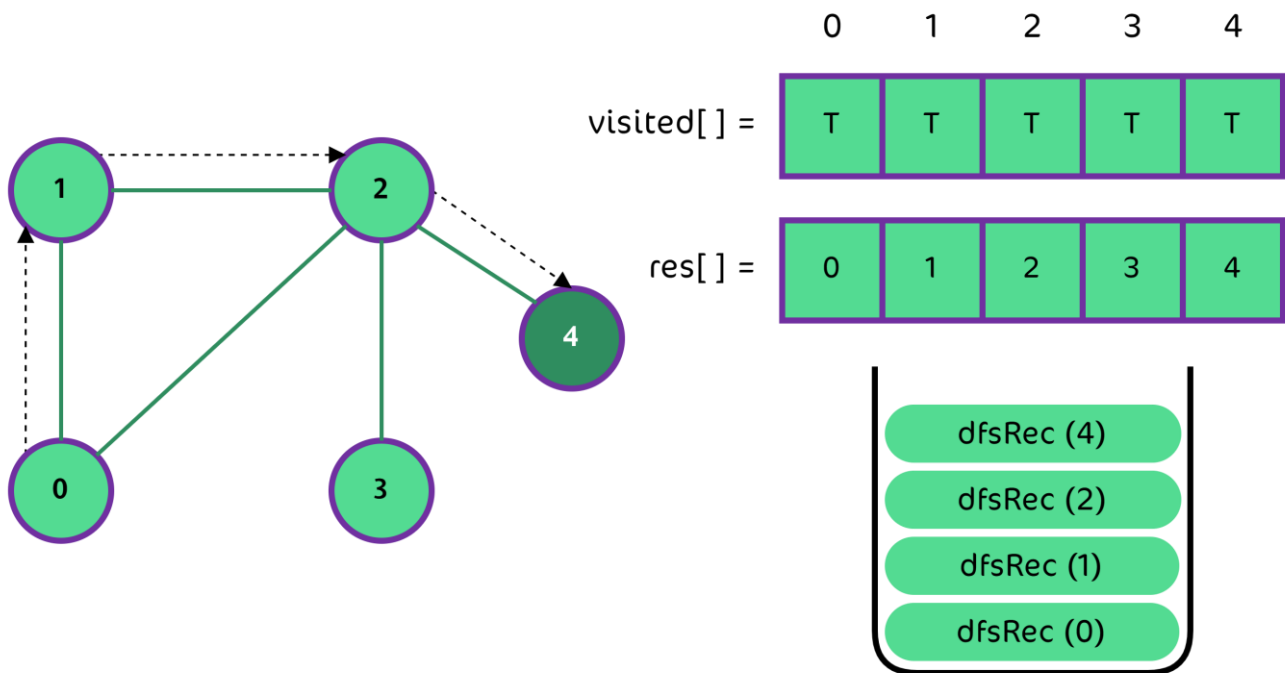
0	1	2	3	
---	---	---	---	--



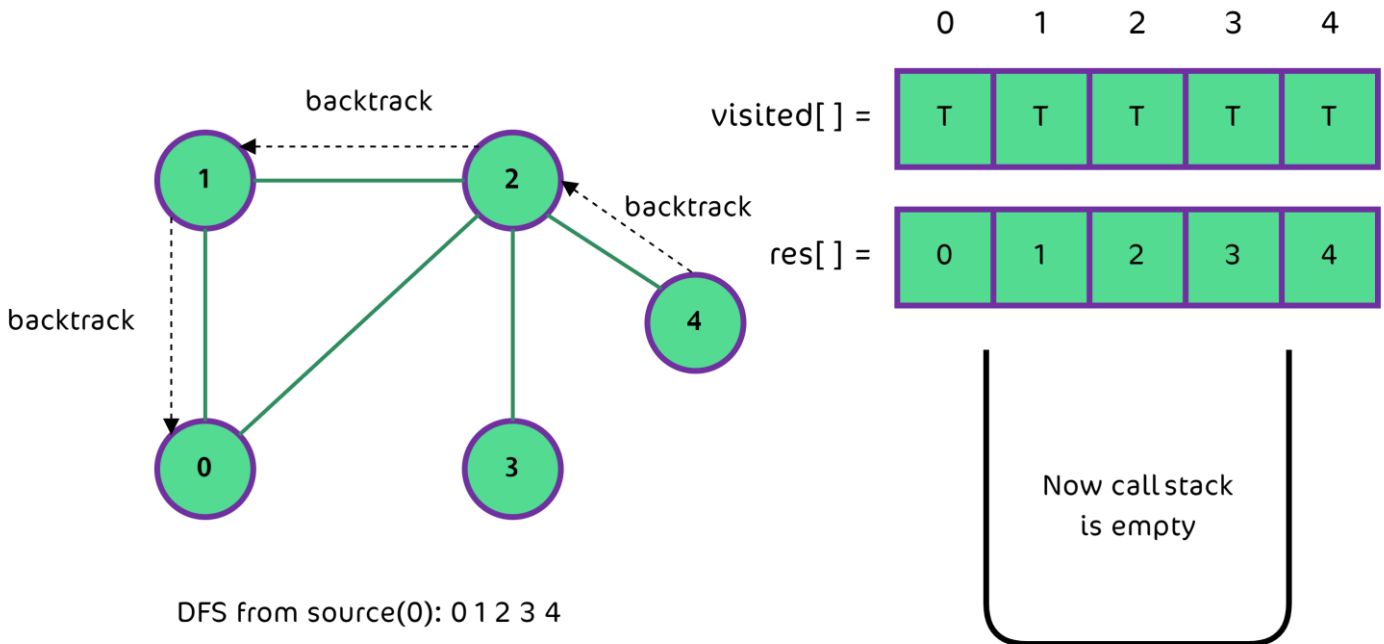
التراجع (Backtracking) من العقدة 3؛ نظراً لعدم وجود جيران غير مُزارين لها، نعود إلى العقدة السابقة (العقدة 2) في مكدّس الاستدعاءات.



نقوم بزيارة الجار الآخر غير المزار للعقدة 2، وهو العقدة 4، يتم استدعاء الدالة لها dfsRec(4)، وتعليمها كعقدة مُزارّة وإضافتها للنتيجة.



نقوم بالتراجع على طول المسار وصولاً إلى عقدة المصدر. وبما أن جميع العقد قد تمت زيارتها بالفعل، فلن يتم إجراء أي استدعاءات أخرى لـ DFS، ويصبح مكدس الاستدعاءات فارغاً.



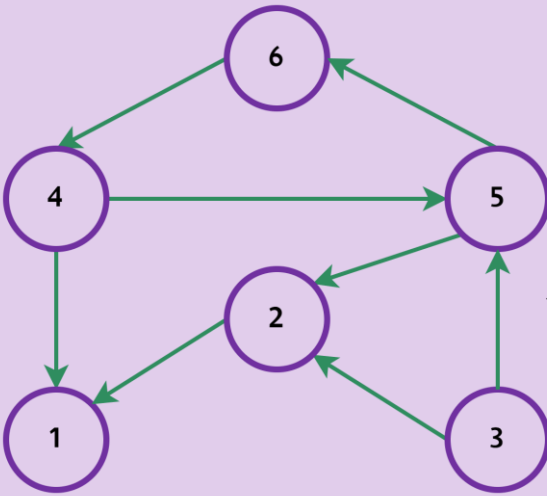
ترتيب المرور النهائي لـ DFS من المصدر (0) هو: 0, 1, 2, 3, 4.

التجول بطريقة المسح بالعرض (BFS)

تخيل أنك ألقيت حجراً في ماءٍ راكد، كيف تتشكل الأمواج؟
تتسع على شكل دوائر تغطي القريب ثم الأبعد فالأبعد.

خوارزمية BFS تعمل كالموجة؛ هي تقول: "سأزور جميع جيراني المباشرين أولاً (المستوى الأول)، وعندما أنتهي منهم تماماً، أنتقل إلى جيران جيراني (المستوى الثاني)، وهكذا"
لهذا تسمى البحث بالعرض أو بالطبقات (Level-Order).
تعتمد BFS بشكل أساسي على الطابور (Queue) الطابور يتبع مبدأ (FIFO) First In, First Out وهو ممتاز هنا لأننا نريد معالجة العقد التي اكتشفناها أولاً قبل العقد الجديدة.

مثال: ليكن لدينا الـ Graph التالي:



- نطبق BFS على العقدة (3).
- أولاً نمسح مجاورات هذه العقدة وهي الـ (5) و (2)، أي أن العقد المُرارة أصبحت [3, 5, 2].
- ننتقل الآن إلى مجاورات الـ (5) وهي العقدة (2) التي قد تمّ زيارتها مسبقاً، والعقدة (6)، فأصبحت العقد المُرارة [3, 5, 2, 6].
- بعدها ننتقل إلى مجاورات الـ (2) وهي الـ (1)، فأصبحت العقد المُرارة [3, 5, 2, 6, 1].
- بعدها نذهب إلى مجاورات الـ (6) وهي الـ (4)، فأصبحت العقد المُرارة [3, 5, 2, 6, 1, 4].
- وهكذا قد قمنا بمسح الـ Graph كاملاً.

Pseudocode Implementation:

في البداية عرّفنا مصفوفة للعقد المُرارة، و queue من أجل حفظ العقد التي لم يتمّ زيارة مجاوراتها بعد.

```

1  boolean visited[n];
2  queue q;
3  void BFS(boolean G[n][n], int node){
4      q.enqueue(node);
5      visited[node] = true;
6      while (not q.isEmpty()){
7          node = q.dequeue();
8          print(node);
9          for(i = 1 to n)
10             if(g[node][i] and visited[i] == false){
11                 q.enqueue(i);
12                 visited[i] = true;
13             }
14         }
15     }

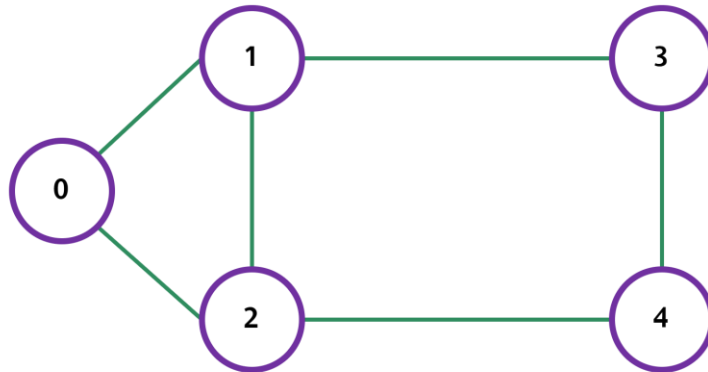
```

لاحظ أن:

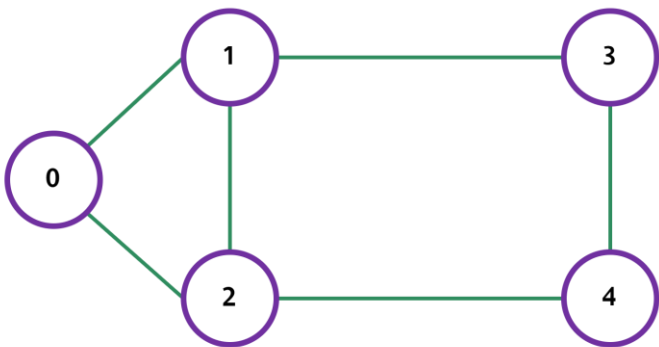
dequeue ترجع العنصر الأول ثم تحذفه، وبما أننا نريد أن نبحث عن مجاورات أول عنصر في الـ queue لذلك نخزن العنصر في node، لنبحث عن مجاوراته لاحقاً، وعند إضافة مجاوراته إلى الـ queue يكون العنصر قد حُذِفَ منها.

بالنظر إلى الرسم البياني التالي (Graph)،

أوجد ترتيب المرور عبر العقد باستخدام خوارزمية البحث بالعرض (BFS Traversal):



قم بإنشاء مصفوفة لتهيئة العقد المُزارة [visited] وطابور انتظار queue لتتبع العقد التي سيتم معالجتها، بالإضافة إلى مصفوفة النتيجة [res].



visited[] =

--	--	--	--	--

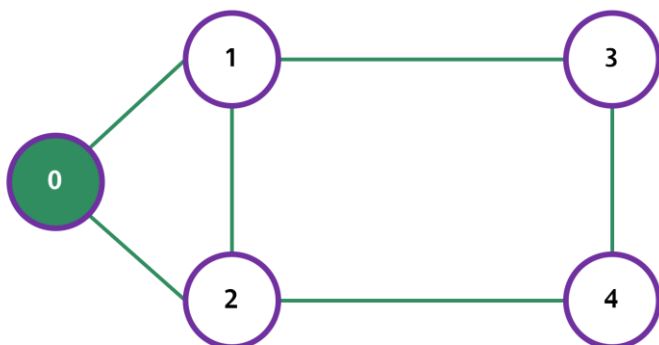
res[] =

--	--	--	--	--

queue

--	--	--	--	--

ابدأ من عقدة المصدر (الرأس 0) وقم بإضافتها داخل طابور الانتظار (queue)، مع تعليمها كعقدة مُزارة وحفظها في النتيجة.



visited[] =

T				
---	--	--	--	--

res[] =

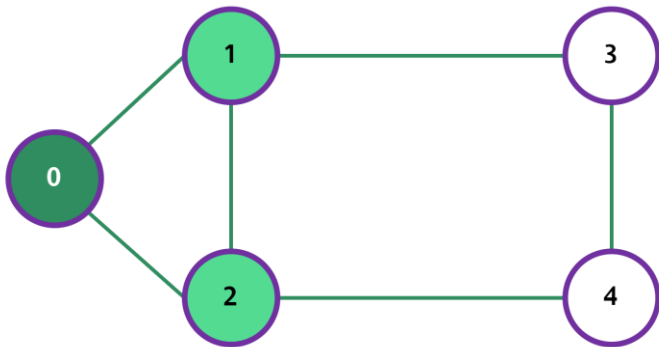
0				
---	--	--	--	--

queue

0				
---	--	--	--	--

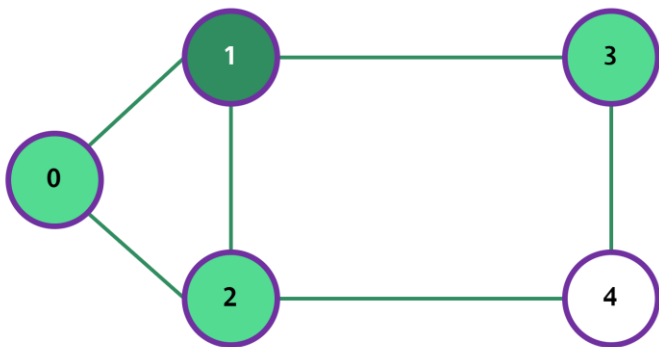
↑
front

قم بمعالجة العقدة 0، ثم أدخل جيرانها غير المُزارين (العقدتين 1 و 2) إلى طابور الانتظار.



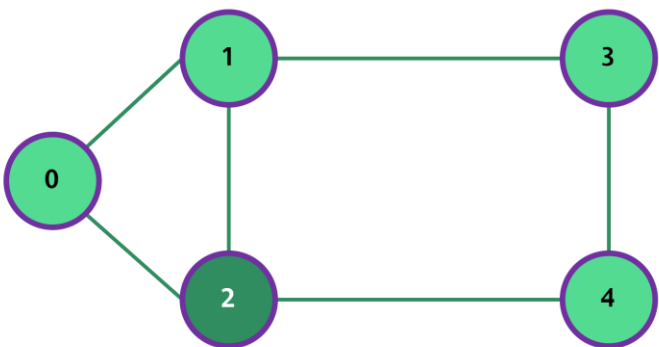
visited[] =	T	T	T		
res[] =	0				
queue	1	2			
	↑				
front					

قم بمعالجة العقدة 1 (الموجودة في مقدمة الطابور)، وأضف جيرانها غير المُزارين (العقدة 3) إلى طابور الانتظار.



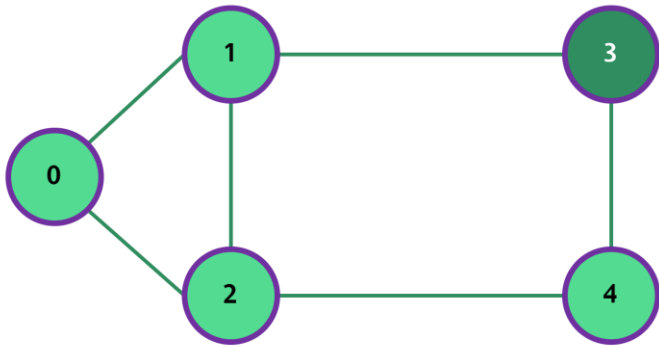
visited[] =	T	T	T	T	
res[] =	0	1			
queue	2	3			
	↑				
front					

قم بمعالجة العقدة 2، وأضف جيرانها غير المُزارين (العقدة 4) إلى طابور الانتظار.



visited[] =	T	T	T	T	T
res[] =	0	1	2		
queue	3	4			
	↑				
front					

قم بمعالجة العقدة 3، وأدخل جيرانها غير المزارين إلى طابور الانتظار (بما أن العقد المجاورة مُزارّة بالفعل، لا يتم إضافة شيء جديد).



visited[] =

T	T	T	T	T
---	---	---	---	---

res[] =

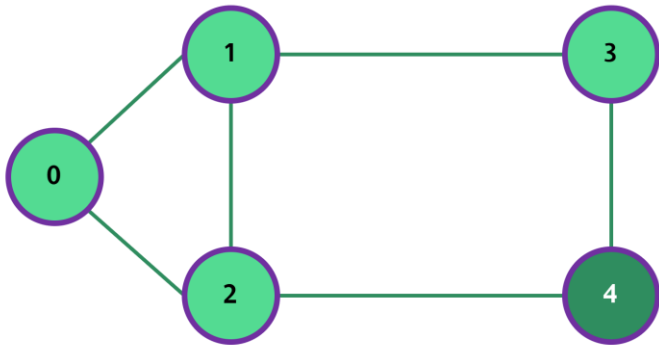
0	1	2	3	
---	---	---	---	--

queue

4				
---	--	--	--	--

↑
front

قم بمعالجة العقدة 4، وأدخل جيرانها غير المزارين إلى طابور الانتظار.



visited[] =

T	T	T	T	T
---	---	---	---	---

res[] =

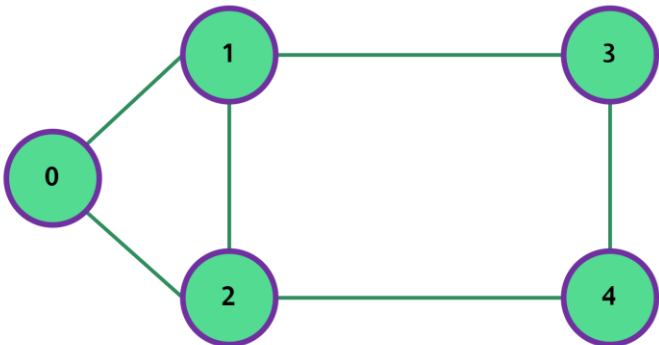
0	1	2	3	
---	---	---	---	--

queue

4				
---	--	--	--	--

↑
front

لقد تمت زيارة جميع جيران العقدة 4 بالفعل، لذا انتقل إلى العقدة التالية في الطّابور. وبما أن الطّابور أصبح فارغاً الآن ولا توجد عقد أخرى لمعالجتها، فإننا نحصل على الترتيب النهائي لخوارزمية BFS.



visited[] =

T	T	T	T	T
---	---	---	---	---

res[] =

0	1	2	3	4
---	---	---	---	---

queue

--	--	--	--	--

ترتيب المرور النهائي لـ BFS هو: 0, 1, 2, 3, 4.

ملاحظة هامة جداً:

- عندما نضيف جيران العقدة غير المُزارين إلى الطّابور فإننا نضعهم في نهاية الطّابور حتّى نتمكّن من معالجة العقدة الأساسيّة أولاً لتحقيق مفهوم الـ BFS بشكلٍ صحيح.
- بما أنّ زيارة العقدة تتمّ لمرة واحدة فقط، فإنّ تعقيد خوارزمية BFS و DFS هي من رتبة $O(n)$.
- في الـ DFS نقوم بعدّة استدعاءات عوديّة و ربما كثرة الاستدعاءات تسبب ظهور مشكلة من نوع StackOverflow، و لحلّ المشكلة يمكننا أن نحلّها بطريقة غير عوديّة (بناء الـ Stack يدوياً ونقوم بعمليات push, pop)...
- عند طباعة العقد في Graph ما بطريقة DFS فإنّ العقد نفسها ستظهر عند استخدام طريقة BFS لكن بترتيب مختلف.

أمثلة واقعية

أبرز استخدامات الـ DFS:

- حلّ المتاهات و الـ puzzle، كشف الحلقات المغلقة (Cycle Detection)، والتّرتيب الطّوبولوجي.

أبرز استخدامات الـ BFS:

- إيجاد أقصر مسار (Shortest Path) في المخطّطات غير الموزونة، وشبكات التّواصل الاجتماعيّ (الأصدقاء المقترحون) و محرّكات البحث، و أنظمة الملاحة (GPS).

هدف بدون خطة هو مجرد أمنية...

